AN5185

Application note

How to use STMicroelectronics firmware upgrade services for STM32WB MCUs

Introduction

This document describes the firmware upgrade services (FUS) available on STM32WB series microcontrollers. These services are provided by STMicroelectronics code, which is located in a secure portion of the embedded flash memory, and are used by any code running on the Arm® Cortex®-M4 processor, or through embedded bootloader commands (also running on the Arm® Cortex®-M4 processor).

1 General information

This document applies to STM32WB series Arm®-based devices.

An **STMicroelectronics wiki page** concerning FUS is available **for additional detailed information** at https://wiki.st.com/stm32mcu/wiki/Connectivity:STM32WB_FUS.

Note: *Arm and Cortex are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.*

The Arm word and logo are trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved.

arm

1.1 Firmware upgrade services definition

FUS (firmware upgrade services) is a firmware running on STM32WB Arm® Cortex®-M0+ and offering the following features:

1. Installation, upgrade, or deletion of the STM32WB Arm® Cortex®-M0+ wireless stack:
   - Only encrypted and signed by STMicroelectronics.
   - Optional: can be additionally double-signed by the customer, if necessary.
2. FUS self-upgrade:
   - Only encrypted and signed by STMicroelectronics.
   - Optional: can be additionally double-signed by the customer, if necessary.
3. Customer authentication key management:
   - Used for double-signing images (can be wireless stack or FUS images).
   - Install, update, and lock the customer authentication key.
4. User key services:
   - Store customer keys
     - Master key
     - Simple clear key
     - Encrypted key (by master key)
     - In a secure area, accessible only by Arm® Cortex®-M0+ code.
   - Write stored key (simple or encrypted) into AES1 (advanced encryption standard) in secure mode (the Arm® Cortex®-M4 cannot access the key).
   - Lock a stored key to prevent its usage until the next system reset.
   - Unload a previously loaded key from AES to prevent its usage by other applications.
   - Key width: 128 or 256 bits.
   - Up to 100 user keys (encrypted by master key or clear) and one user master key.
5. Communication with Arm® Cortex®-M4 (user code or bootloader):
   - Through the IPCC commands and the response model, which is the same as the wireless stack model.
   - Commands already supported by the STM32WB system bootloader.

1.2 FUS versioning and identification

To identify the FUS version, read the shared tables memory in SRAM2a, as explained in Section 1.6: Shared tables and memory usage and Section 6.1: Shared tables usage.

The first word in SRAM2a pointed by IPCCDBA Option Bytes is the "Device info table" address. This table (described in Table 9) contains the FUS version at offset 0xC, which is encoded on four bytes. Typically, if IPCCDBA = 0x0000 and @0x20030000 contains 0x20030024, then the FUS version is @0x20030030.

Installation of a FUS image must follow the conditions stated in the image binary release notes.

Note: When using the SWD interface with the STM32CubeProgrammer (STM32CubeProg) older than V2.7.0, the address of the device information table is located at 0x20030890. For STM32CubeProgrammer V2.7.0 and higher, the device information table is located at 0x20030024.

Table 1. FUS versions

Version	Description
v0.5.3	The default version is programmed in production for all STM32WB5xx devices. It must be upgraded to v1.0.1 on STM32WB5xG devices or to v1.0.2 on STM32WB5xE/5xC devices. This version is not available for download on www.st.com and it cannot be installed by users.
v1.0.1	First official release available on www.st.com and dedicated to STM32WB5xG devices only (1 Mbyte flash memory size) This version must not be installed on STM32WB5xE/5xC devices, otherwise the device enters a locked state and no further updates are possible.
v1.0.2	First official release available on www.st.com and dedicated to STM32WB5xE/5xC devices (512 Kbytes and 256 Kbytes flash memory size) Use the v1.0.2 on the STM32WB5xG devices if the devices present FUS V0.5.3. If an STM32WB5xG device has FUS v1.0.1, then there is no need to upgrade to v1.0.2, as it does not bring any new feature/change vs. v1.0.1. In case FUS v1.0.2 installation is started by a user on an STM32WB5xG device with FUS v1.0.1, FUS returns FUS_STATE_IMG_NOT_AUTHENTIC error and discard the upgrade.
v1.1.0	FUS update to support the following features: • Add FUS_ACTIVATE_ANTIROLLBACK command that allows activating Anti-rollback on wireless stack by user. User can activate this feature to prevent any installation of older wireless stack. • Replace Safeboot by v1.1.0 version (replace full chip lock by factory reset) • Add factory reset in case of Flash ECC, corruption, or Option Bytes corruption error. Factory reset means erase of wireless stack if present and reboot on FUS and full erase of other user sectors. FUS v1.1.0 can be installed only on devices containing v1.0.1 or v1.0.2 FUS. In case a device has v0.5.3 installed, the user must first install v1.0.2 then install v1.1.0. When installing FUS v1.1.0 over an FUS v0.5.3 results in FUS_STATE_IMAGE_NOT_AUTHENTIC error and discarding the upgrade.
v1.1.1	FUS update to support STM32WB5xx 640 KB products. This version is not available on www.st.com, and cannot be used for upgrade. This version is fully compatible with v1.1.0, and does not present any differences, except management of the new 640 KB products.
v1.1.2	FUS update to: • optimize the flash usage. Making it possible to install a stack, maintaining a one sector separation below a previously installed stack (instead of the stack size space constraint explained in Section 2: Wireless stack image operations) • provide security enhancements. To upgrade from FUS v1.1.0 to FUS v1.1.2 , the Anti-rollback must first be activated. Before activating Anti-rollback, a wireless stack installed must be present. Upgrading from v1.1.0 to v1.2.0 is possible without constraints nor additional operations from the user.

Version	Description
v1.2.0	FUS update to: • Includes v1.1.2 FUS updates in production • Allows direct update from FUS v1.1.0 to FUS v1.2.0 without activating the Anti-rollback. • Allows direct update from FUS v0.5.3 to FUS v1.2.0 (without installing intermediate FUS versions) • Security updates Upgrading from FUS v1.1.0 or any other FUS version, to FUS v1.2.0 is possible without constraints and no interaction from the user.
v2.0.0	FUS update to: • Adding data migration feature during wireless stack update (to preserve BLE bonding/ pairing data during the update) • Robustness improvement to resets • Security updates
v2.1.0	FUS update to: • Robustness and security improvement
v2.2.0	FUS update to: • Enhanced Security, SA0024 remains applicable

The table below details the FUS version compatibility options (when it is possible to upgrade from a version to another). FUS v1.2.0 is the version that allows the upgrade from any previous version. It is released in two binaries:

- `stm32wb5x_fus_fw.bin`: when FUS V1.2.0 is installed, it can be updated to latest FUSv2 version
- `stm32wb5x_fus_fw_V1.2.0.bin`: for upgrades from any FUS version V1.x.y when x<2 to version V1.2.0
- `stm32wb5x_fus_fw_for_fus_0_5_3.bin`: upgrades from FUS version V0.5.3 to FUS V1.2.0

Table 2. FUS version compatibility

Legend:

- X: Cannot upgrade.
- √: Upgradable.
- **X**: Must not be upgraded. Otherwise, the encryption keys are lost.
- (1): Upgradable but a Bluetooth® LE stack needs to be installed first and Anti-rollback enabled.
- (2): Upgradeable with dedicated FUS v1.2.0 binary version for FUS v0.5.3.

Upgrade		To								
		v0.5.3(1)	v1.0.2(1)	v1.1.0(1)	v1.1.1(1)	v1.1.2 (1)	v1.2.0	v2.0.0	v2.1.0	v2.2.0
From	v0.5.3(1)	X	√	X	X	X	(2)	X(2)	X(2)	X(2)
	v1.0.2(1)	X	X	√	X	√	√	X(3)	X(3)	X(3)
	v1.1.0(1)	X	X	**X**	X	(1)	√	X(3)	X(3)	X(3)
	v1.1.1(1)	X	X	X	X	√	√	X(3)	X(3)	X(3)
	v1.1.2 (1)	X	X	X	X	√	√	X(3)	X(3)	X(3)
	v1.2.0	X	X	X	X	X	√	√	√	√
	v2.0.0	X	X	X	X	X	X	√	√	√
	v2.1.0	X	X	X	X	X	X	X	√	√
	v2.2.0	X	X	X	X	X	X	X	X	√

1. *Not available on STM32WB1x*
2. *Not directly upgradable, an upgrade to FUS v1.2.0 must be made first by using the dedicated binary for FUS v0.5.3.*
3. *Not directly upgradable, upgrade to FUS v1.2.0 first.*

A FUS version is available from two different sources:

- Programmed directly in the STM32WB series devices by STMicroelectronics during the production phase.
- Available from www.st.com. This method is used mainly for the FUS version upgrade process.

The following table details the availability of each version at production and on www.st.com.

Table 3. FUS versions availability

Legend:

- X: Not available
- √: Available

Version	Production	Binary on www.st.com
v0.5.3	√	X
v1.0.2	√	√
v1.1.0	√	√
v1.1.1	√	X
v1.1.2	X	√
v1.2.0	√	√
v2.0.0	X	√
v2.1.0	X	√
v2.2.0	√	√

1.2.1 Known limitations

This section details the known limitations on the FUS version (V1.2.0).

- Upgrade error
  Possible upgrade error in the case of external power loss or reset events. This is in the event of external power loss, or forced reset during the firmware upgrade operation. The ongoing operation might be corrupted. In that case FUS abort the operation and return an error message. Workaround: when an error message is returned, repeat the firmware upgrade operation from the beginning. Special care must be taken in the case of over-the-air (OTA) upgrades, where the wireless stack might be needed to download the image again.
- Wireless stack
  In the particular case when a stack B has to be installed on top of an already installed stack A, and if stack B is larger than stack A by exactly one sector, FUS rejects the operation (returns an error message) unless stack B is loaded at an address "add" `< 0x080F4000 - 3x SizeOf("B")`. The workaround is to add padding to the wireless stack B to avoid the condition of one sector difference in size. This limitation can be seen when using some wireless stack present on STM32CubeWB V1.14.0. For more details, refer to the STM32CubeWB release notes. From STM32CubeWB v1.14.1 release, the size of the wireless firmware binaries is controlled to guarantee at least two sectors size difference between all generated binaries to workaround the limitation.

The latest FUSv2 version installation fixes these limitations and improves robustness and security.

1.3 How to activate FUS

The FUS runs on Arm® Cortex®-M0+ and on the protected flash memory zone dedicated for FUS and wireless stack. There are two possible situations:

Table 4. FUS activation cases

Situation	How to activate FUS
No wireless stack is running (for example the first time the STM32WB series device is running or the wireless stack has been removed)	• Activate the CRC Clock through the RCC register. For instance `__HAL_RCC_CRC_CLK_ENABLE()`. • Ensure Arm® Cortex®-M0+ is activated by setting the C2BOOT bit in the PWR_CR4 register. • Ensure IPCCDBA (Option Bytes) points to a valid shared table information structure in SRAM2a. Enter the correct pointers to the device information table and system table. *Note: The above steps are performed automatically by the system bootloader. So, if device boot is configured on system memory, the FUS must be activated, with no need for further user actions. Otherwise, these actions must be performed by the user code running on a Arm® Cortex®-M4 CPU.*
The wireless stack is installed and running	Perform the same steps as above. Request a wireless stack to launch FUS by sending two consecutive FUS_GET_STATE commands. The first one must return FUS_STATE_NOT_RUNNING state, the second causes FUS to start.

To check if FUS is running or not, the following options are available:

- Send a single FUS_GET_STATE command and check the return status. If it is FUS_STATE_NOT_RUNNING, then FUS is not running.
- Check the SBRV Option Bytes value:
  - On WB5x as an example, if it is 0x3D800 (for FUS V0.5.3) or 0x3D000 (for FUSv1.x.y or FUSv2.x.y) then, FUS must be running.
  - If it is different from 0x3D800 (for FUS v0.5.3) and from 0x3D000 (for FUS v1.x.y and FUSv2.x.y) then FUS is not running.
- Send a wireless stack command:
  - If it is acknowledged, then FUS is not running
  - If it is not acknowledged, then FUS is running
- Read the shared table information:
  - Read IPCCDBA (in Option Bytes) to get the shared tables start address in SRAM2a
  - Get the device information table address
  - Read the field “Last FUS active state”
    - 0x04 means that the stack must be running
    - Other values mean that FUS must be running
  - Read the "Async Ready" event that is sent by FUS at startup. For more information about this event and content, refer to Section 6.3.2: Event packet.

1.4 Memory mapping

The FUS has a dedicated space in the flash memory that depends on the FUS size. It also uses a dedicated space in SRAM2a and SRAM2b, and a shared space in SRAM2a (shared tables). The size of the dedicated space in the flash memory, SRAM2a and SRAM2b is defined by Option Bytes. For more information, refer to the product reference manual.

The dedicated flash memory and SRAM areas are shared with the wireless stack if it is installed. But at a given time, either the FUS or the wireless stack is running on Arm® Cortex®-M0+.

Figure 1. Flash memory mapping

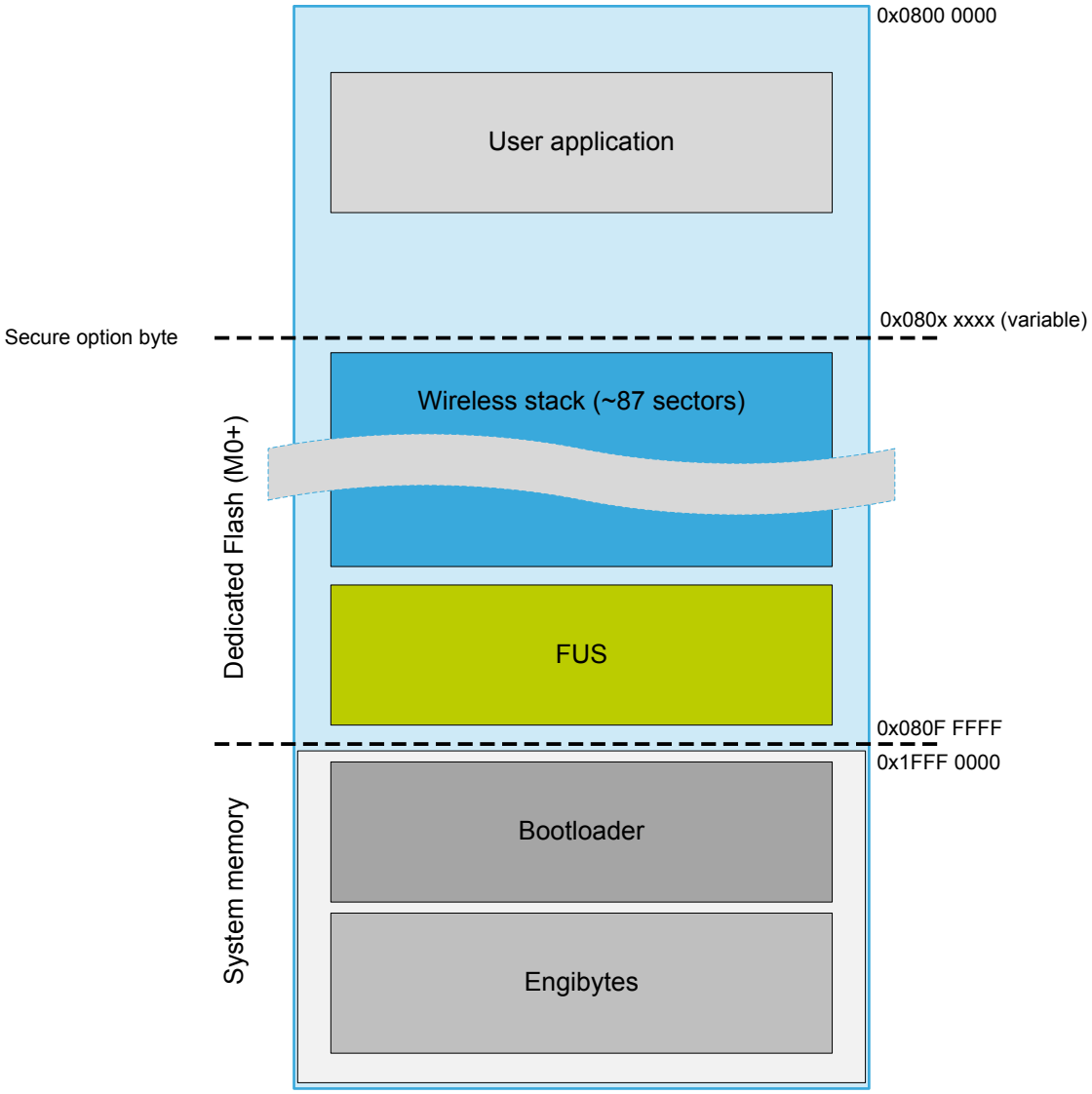


Figure 2. **SRAM memory mapping**

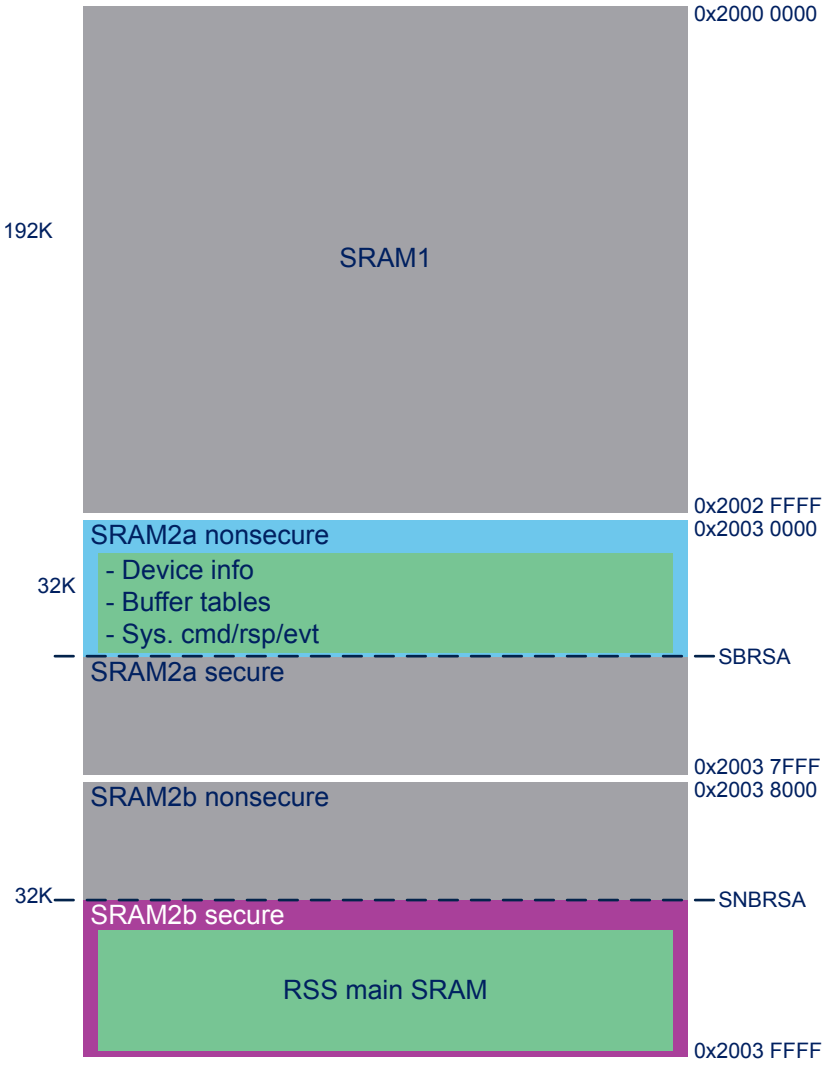


Table 5. **Main characteristics difference between STM32WB5x/WB3x and STM32WB1x with respect to FUSv2**

Item	STM32WB5x/WB3x	STM32WB1x
Sector size	4 Kbytes	2 Kbytes
SRAM2a start address	0x20030000	0x2003000
SRAM2a size	32 Kbytes	32 Kbytes
SRAM2b start address	0x20038000	0x20038000
SRAM2b size	32 Kbytes	4 Kbytes
FUS start address	0x080F4000	0x08046000
SBRV by default (FUS running)	0x3D000	0x11800
SFSA by default (FUS running and no stack installed)	0xF4 (1)	0x8C(2)
FUS version address	0x20030030	0x20030030

Item	STM32WB5x/WB3x	STM32WB1x
External interface that can be used to communicate with FUS	System bootloader USB DFU System bootloader USART SWD (FUS operator)	USART SWD (FUS operator)
Internal interface that can be used to communicate with FUS	IPCC (M4 application)	IPCC (M4 application)
UFB bank address	BANK1: 0x080FB000 BANK2: 0x080FC000	BANK1: 0x0804C000 BANK2: 0x0804C800
CKS feature	YES	NO

1. *Except for FUS v0.5.3, where SFSA is 0xF6. When the data area is 16 Kbyte NVM for versions FUSv2.x.y, SFSA is 0xF0 for STM32WB5xxG (1M). Note that the default SFSA value can vary between STM32WB flash size derivatives.*
2. *When the data area is 16 Kbyte NVM for versions FUSv2.x.y, SFSA is 0x84 for STM32WB1x.*

Table 6. Additional device characteristics for different STM32WB flash derivatives with respect to FUSv2

Characteristics	STM32WB5xxG (1 M)	STM32WB5xxY (640 K)	STM32WB5xxE (512 K) STM32WB3x (512 K)	STM32WB1x (320 K)	STM32WB5xxC (256 K) STM32WB3x (256 K)
FUS address	0x080F4000	0x080F4000	0x080F4000	0x08046000	0x080F4000
SFSA_Prod	0xF4	0xA0	0x080	0x8C	0x40
Sector_size (bytes)	0x1000	0x1000	0x1000	0x800	0x1000
Flash_size (kB)	0x400	0x280	0x200	0x140	0x100

1.5 FUS resource usage

The FUS only configures/uses the resources listed in Table 7.

The RCC (reset and clock control), flash memory, PWR (power control) and all necessary components for the STM32WB series microcontroller normal operation must be configured by Arm® Cortex®-M4 application prior to enabling Arm® Cortex®-M0+. If the system bootloader is used, it performs this initialization.

Table 7. FUS resource usage

Resource	Case	Configuration
Flash	Always	A dedicated flash memory area is used by the FUS depending on its size, and on the size of the current wireless stack and the new wireless stack image requested to be installed. Parts of the dedicated flash memory may be written and/or erased during FUS operations. **Caution:** *Arm® Cortex®-M4 user code shall not perform write or erase operations while the FUS is executing its operations.*
SRAM2b	Always	SRAM2b secure area is used by FUS depending on its version.
SRAM2a	Always	SRAM2a secure area is used by FUS depending on its version. SRAM2a public area is used by FUS to write into shared tables for information table and commands table.
IPCC	Always	IPCC is used by FUS for mail boxing between Arm® Cortex®-M0+ and Arm® Cortex®-M4 user application or bootloader or JTAG. Two channels are used: P1CH2 (command/response channel) and P1CH4 (trace channel).
PKA	When install is requested	PKA is enabled, configured, and used for signature verification.
AES1	When a key service is requested	AES1 is configured in secure mode (key register is accessible only by Arm® Cortex®-M0+) AES1 key register is written by FUS with the key requested by the user. Once AES1 is configured in secure mode, it remains in secure mode until the next system reset, or FUS_UNLOAD_USR_KEY command is executed.
Option Bytes	When install/ delete is requested	Option Bytes are programmed by FUS using Arm® Cortex®-M0+ registers: only the SFR and SBRR registers are modified.
CRC	When install is requested and CPU reboot on FUS	CRC is used for authentication and it is not initialized by FUS. If CRC is used by a Arm® Cortex®-M4 user application, it has to be reset before starting FUS or wireless stack install operations.
System Reset	When install/ delete is requested	CRC is used for authentication purposes. If the CRC peripheral is also used by an Arm® Cortex®-M4 user application, it must be reset before starting FUS or wireless stack install operations to avoid conflicts. However the CRC peripheral is used by FUS not only when a stack install or upgrade is requested, but it is also reinitialized by FUS each time CPU2 reboots in FUS mode, (for example, via LL_PWR_EnableBootC2()). Therefore, CRC registers are modified immediately when CPU2 starts in this mode.
NVIC	Always	The following handlers are used: • NMI • SysTick • IPCC_C2_RX_C2_TX_HSEM

Important: *During FUS or wireless stack upgrade/delete operations, Arm® Cortex®-M4 and SWD shall not:*

- *Perform any write/erase operation on flash memory.*
- *Perform any write on Option Bytes.*
- *Change PWR and RCC configuration.*

If any of the above operations are performed during FUS or wireless stack upgrade/delete, there is a risk of corrupting the flash memory and losing data.

Important: In case power supply failure occurs during a FUS operation (install/delete), one of the following three cases may occur:

- *Power failure without impact: if the flash memory content is not corrupted, FUS recovers the failure and continues operating without the need for any user intervention.*
- *Power failure with flash memory corruption: the flash memory content is corrupted. The image is not installed by FUS (rejected as noninteger). FUS erases the image and generates an error (FUS_ERR_IMG_CORRUPT). The user must restart the whole operation by reloading the binary and send an upgrade command to FUS.*
- *Power failure with option bytes corruption: safeboot is started by hardware and the entire flash memory is locked by hardware. In this case, if FUS v1.1.0 or higher version is running, then a factory reset is triggered (the user must activate CM0 by writing the value 0x00008000 at the address @0x5800040C). If a FUS version lower than v1.1.0 is running then no recovery is possible at this point.*

Note: If there are user keys stored by FUS, when FUS is upgraded, these keys are erased.

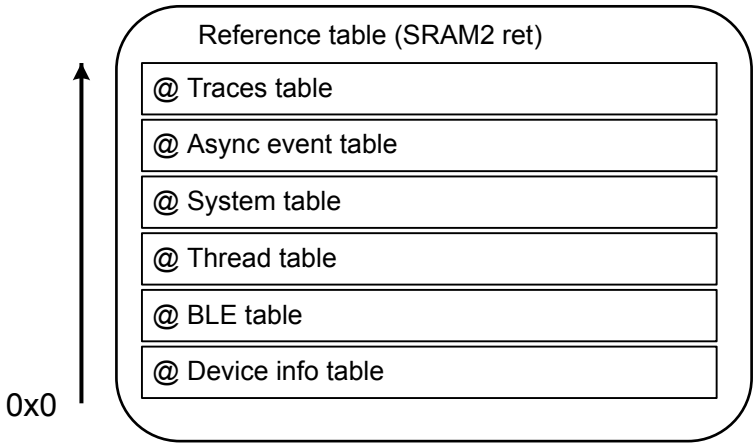
1.6 Shared tables and memory usage

Communication data buffers are pointed to by a lookup table for which the address is determined by an Option Byte: IPCCDBA (IPCC mailbox data buffer base address). This address provides the base address of the buffer table pointers, as detailed in the application note *How to build wireless applications with STM32WB MCUs* (AN5289).

If IPCCDBA points to an address that does not fit all table pointers, such as (SRAM2a_END_ADDRESS - SharedTable_BaseAddress) < SizeOf(SharedTable), the FUS must discard usage of shared table completely and thus no communication or commands are possible with the FUS.

The user application has to set up the shared table base address correctly, otherwise it must stop the FUS services initialization.

Figure 3. Shared table architecture



FUS uses only two tables:

- Device information table: this table provides useful information from FUS to the Arm® Cortex®-M4 user application (or JTAG) at startup (content written by FUS at startup).
- System table: this table allows the exchange commands and responses between FUS and Arm® Cortex®-M4 user application.

2 Wireless stack image operations

The FUS allows the user to install, upgrade, and delete the wireless stack.

The wireless stack has to be provided by STMicroelectronics (encrypted and signed) to be installed by the FUS. The user may add a custom signature to the wireless stack image binary using the STMicroelectronics tools, as detailed in Section 4: User authentication, if the user authentication key has already been loaded by FUS.

The wireless stack install, upgrade, and delete operations are performed through the bootloader, JTAG, or user application. STM32CubeProgrammer provides the tools to perform these operations through the bootloader interfaces: USART and USB-DFU, and also directly through the SWD interface.

2.1 Wireless stack install and upgrade

This document uses two definitions:

- Wireless stack install: the first installation on a chip where there is no wireless stack already installed.
- Wireless stack upgrade: the installation on a chip where a wireless stack is already installed (may be running or not).

Operation instructions

To perform a wireless stack install or upgrade, follow the procedure below:

1. Download the wireless stack image from www.st.com or from the STM32CubeMX repository.
2. For FUSv1.x.y with x≤2 and legacy stack format, when using Cubeprogrammer in manual mode, write the wireless stack image to the user flash memory at the following address: DownloadAddress = 0x08000000 + (SFSA x SectorSize) - ImageSize – (1 x SectorSize).
   For FUSv2.x.y with x≥0 supporting both legacy and new stack formats and considering all flash-size derivatives, the wireless stack start address can be computed as:
   – Legacy format: DownloadAddress = min(0x08000000 + (FlashSize x 1KB), 0x08000000 + (SFSA x SectorSize)) - ImageSize
   – New format: DownloadAddress = min(0x08000000 + (FlashSize x 1KB), 0x08000000 + (SFSA x SectorSize)) – ImageSize –NvmSectionSize x SectorSize

   where:
   – DownloadAddress is the address where the new wireless stack is loaded, aligned to sector size.
   – SFSA is the Secure Option register value indicating the current boundary of flash memory secure area.
   – SectorSize is 4 KB for STM32WB5xxx, and 2 KB for STM32WB10xx and STM32WB15xx.
   – ImageSize is the size, in bytes, of the binary to install.
   – FlashSize is the available HW flash size in Kbytes for different flash size derivatives.
   – NvmSectionSize is the size of the reserved NVM flash area containing stack pairing data.

   When using the STM32CubeProgrammer, the best placement is suggested based on SFSA and binary size. In the case where the new wireless stack size is larger than the already installed stack, refer to "Memory instructions" in Section 2.2: Wireless stack delete.
3. Ensure that the FUS is running. To do so, follow the steps in Section 1.3: How to activate FUS.
4. Send the FUS_FW_UPGRADE command through the IPCC mechanism (For further details, refer to the sections below).
5. Send FUS_GET_STATE until a state equal to FUS_STATE_NOT_RUNNING is reached. This means that the wireless stack has been installed and is now running.

During the installation process, expect multiple system resets to occur. These system resets are performed by FUS and are necessary for the modification of dedicated memory parameters and to make Arm® Cortex®-M0+ run the installed wireless stack. The number of system resets depends on the configuration and the location of new and old images.

Note: FUSv2 automatically computes the best start address for the wireless stack installed. If the user downloads the address below the optimized start address, FUSv2 automatically moves the wireless stack at the optimized address. If the user downloads the address above the optimized address, an error is returned.

Table 8 explains possible errors when the install/upgrade operation is requested and their respective results.

Table 8. **FUS upgrade returned errors**

Error	Reason	Result
Not enough space	The memory space between the current installed wireless stack and the address of the loaded image is too small.	Installation request is rejected. FUS returns an error state, and goes back to an idle state.
Image signature not found	Incorrect or corrupted signature header or body.	FUS returns an authentication error, then goes back to idle state. The image is not installed and no changes on flash memory/ SRAM.
Image customer signature not found	Incorrect or corrupted signature header or body.	FUS returns an authentication error, then goes back to an idle state. The image is not installed and no changes on flash memory/ SRAM.
Image corrupted	Incorrect image header or corrupted image.	FUS returns an image corruption error, then goes back to an idle state. The image is not installed, and it is erased by FUS.
No state is returned by FUS	A reset performed by FUS has occurred before receiving the command response.	Command resending must result in receiving a FUS response.
Other failures	External power interruption or external reset during FUS operation	FUS must be able to recover and delete the corrupted image, and go back to its default state. It can perform several system resets to complete the recovery operation.

Pairing data migration

FUS can keep pairing data section during a wireless stack upgrade. To do so, two requirements must be met:

- The FUS version installed must at least v2.0.0.
- Not all wireless stacks support this feature. Compatible wireless stacks are listed in the Release_Notes.html inside the STM32WB_Copro_Wireless_Binaries folder from the STM32CubeWB MCU Firmware Package.

The procedure is the same as the standard wireless upgrade procedure (Operation instructions) except for the download address, as follows.

For FUSv2.x.y with x≥0, write the new format wireless stack image in the user flash memory at the address: DownloadAddress = 0x08000000 + (SFSA x SectorSize) - ImageSize - NvmSectionSize, where:

- DownloadAddress is the address where the new wireless stack is loaded and aligned to the sector size.
- SFSA is the Secure Option register value indicating the current boundary of flash memory secure area.
- NvmSectionSize is the size of the reserved NVM flash area containing stack pairing data.
- ImageSize is the size, in bytes, of the binary to install.

During the process, the FUS upgrades the wireless stack while preserving the pairing data section (except in case of fallback, meaning when an old version legacy stack format is installed to replace a new format stack with defined NVM section outside binary).

Memory considerations

A column *FUSv2 compliant* has been added in the recommended flash address table for Bluetooth® LE stacks and concurrent mode stacks, only to be installed when FUS is upgraded to FUSv2 At first install, or in case there is no wireless stack is installed, the FUS does not make any optimization on the address where the wireless stack is installed. The wireless stack image must be installed at the same address where it has been loaded by the user. With FUSv2, this is no longer the case; the wireless stack is installed by FUS as high as possible in flash memory to avoid losing memory space.

At wireless stack upgrade (a wireless stack is already installed), the FUS may move the upgraded stack after upgrade and before running it.

The remaining space in this case is left free for Arm® Cortex®-M4 user application usage.

Once the install/upgrade operation is successfully completed, the SRAM2a, SRAM2b, flash memory secure boundaries and SBRV values are changed according to the requirements of the installed wireless stack.

2.2 Wireless stack delete

Wireless stack delete removes the wireless stack already installed on a chip, but keep the saved NVM section with pairing data, if any.

Operation instructions

To perform a wireless stack delete operation, follow these steps:

1. Ensure that FUS is running, (follow the steps in Section 1.3: How to activate FUS).
2. Send the FUS_FW_DELETE command through the IPCC mechanism, which is explained in the following sections.
3. Send FUS_GET_STATE until FUS_STATE_IDLE is reached with error code FUS_STATE_NO_ERROR. During the delete process, expect multiple system resets to occur. These system resets are performed by FUS and are necessary for the modification of dedicated memory parameters. The number of system resets depends upon the configuration and the location of the wireless stack.

If no wireless stack is installed and a delete request is sent, then the FUS returns an error state informing that no wireless stack was found (FUS_STATE_IMG_NOT_FOUND).

Memory considerations

After the delete operation is done successfully, all the space used by the wireless stack becomes free for usage by Arm® Cortex®-M4 user applications, or for further wireless stack install operations.

Image start address must be aligned to sector start (this is a multiple of SectorSize), and the image size must be a multiple of 4 bytes, otherwise FUS rejects the installation procedure.

Following limitations are applicable for FUS version ≤v1.x.y and have been corrected with FUSv2. These limitations have been corrected with FUSv2. If a new wireless stack "B" must be installed while a stack "A" is already installed, and the size of B is larger than the size of A, there are two possible options for the address where "B" can be loaded:

- Condition1 (back-to-back option): (Both conditions C1 and C2 below must be met)
  - C1. AddressOf(B) > (FUS_ADD - (2 x SizeOf(B)) (aligned to sector size)
  - C2. AddressOf(B) < AddressOf(A) - SizeOf(B) (aligned to sector size)
- Condition2 (non back-to-back option): (only condition C3 must be met)
  - C3. AddressOf(B) < (FUS_ADD - (3 x SizeOf(B) (address must be aligned to sector size)

Where FUS_ADD is the FUS address in the flash (0x080F4000 for STM32WB5xxx and 0x08046000 for STM32WB1xxx).

For cases using FUSv1.x.y with x≤2, the most optimized download address is:DownloadAddress = 0x08000000 + (SFSA x SectorSize) - ImageSize - 1xSectorSize.

2.3 Wireless stack start

After the installation of a wireless stack, by default it starts running just after installation without the need for any user action. But if the user application sends two consecutive FUS_GET_STATE commands, the wireless stack boots again on FUS. From a user perspective, the wireless stack is installed but not running (FUS is running).

In that case, it is possible to launch the wireless stack execution by sending the FUS_START_WS command. This command switches from the Arm® Cortex®-M0+ execution to the wireless stack, and results in at least one system reset.

The command is completed when FUS_GET_STATE returns a FUS_STATE_NOT_RUNNING value. On receiving this value, no other FUS_GET_STATE must be issued, otherwise the FUS is executed again.

2.4 Anti-rollback activation

When FUS supports Anti-rollback, it is possible to activate this feature by sending a command to the FUS.

When this command is executed by FUS, it is no longer possible to deactivate it.

This feature is executed through the FUS_ACTIVATE_ANTIROLLBACK command.

After sending this command, it is possible to check its status by sending the FUS_GET_STATE command. The FUS then returns the state FUS_STATE_IDLE.

This command is not reversible. This command does not apply to FUS, as no rollback is possible on FUS anyway.

Important: *Before activating Anti-rollback, ensure that a wireless stack is correctly installed, and that it has not been deleted. If it is activated without any wireless stack installed, the FUS registers 0xFFFFFFFF as the new version, and it is not possible to install a wireless stack.*

When Anti-rollback is activated, it locks the version of wireless stack that can be installed.

It is impossible to install any wireless stack with a version lower than the current one. For example, if wireless stack V1.9.0 is installed, when Anti-rollback is activated, only wireless stacks with versions V1.9.0 or higher can be installed (it is no longer possible to install, for example, V1.8.0).

3 FUS upgrade

Upgrading the FUS does not remove the NVM data section reserved, if any. This is only applicable for FUSv2 (see Pairing data migration).

3.1 Operation instructions

To perform a FUS upgrade, perform the following steps:

1. Download the FUS image from www.st.com or from the STM32CubeMx repository.
2. Write the FUS image in the user flash memory at the address indicated in the FUS image directory Release_Notes.html file.
3. Ensure that FUS is running (follow the steps in Section 1.3: How to activate FUS).
4. Send FUS_FW_UPGRADE command through the IPCC mechanism (explained in the sections below).
5. Send FUS_GET_STATE until getting a state equal to FUS_STATE_NOT_RUNNING (this means that the wireless stack has been installed and is now running).

During the installation process, expect multiple system resets, performed by FUS, needed for the modification of dedicated memory parameters and to make Arm® Cortex®-M0+ run the installed wireless stack. The number of system resets depends upon the configuration and the location of new and old images.

FUS identifies the image as the FUS upgrade image and launches the FUS upgrade accordingly. This operation can result in a relocation of the firmware stack if it is already installed, and if the size of the new FUS is larger than the size of the current FUS. This information and any relative constraints are detailed in the FUS image release note.

3.2 Considerations on memory

The FUS upgrade requires no specific memory conditions for the address where the image is loaded. But if the new FUS image size is larger than the existing FUS size, the upgrade can move the wireless stack lower in flash memory, to ensure enough space is available for the FUS upgrade.

This means that:

- Less flash memory is available for the Arm® Cortex®-M4 user application.
- The wireless stack is moved from its current address to another one, defined by FUS.
- If a user code is written in the sectors close to the wireless stack start sector, there is a risk of it being erased during this operation.

The size of the FUS and the results of its upgrade are detailed in its Release_Notes.html file.

The image start address must be aligned to a sector start (a multiple of the sector size), and the image size must be a multiple of 4 bytes, otherwise FUS rejects the installation procedure.

4 User authentication

The FUS services allow the user to add a customized signature to any image (wireless stack or FUS image) provided by STMicroelectronics (encrypted and signed by STMicroelectronics).

The instruction to sign a binary with a user authentication key are provided in the STM32CubeProgrammer user manual.

FUS checks on the user signature only if a user authentication key has already been installed.

The signature is a 64-bytes data buffer based on RSA ECC Prime256v1 (NIST P-256) and HASH-256. It is generated by the STM32CubeProgrammer tool.

4.1 Install user authentication key

FUS allows storing a user authentication key through the following steps:

1. Ensure that FUS is running (follow the steps in Section 1.3: How to activate FUS).
2. Send FUS_UPDATE_AUTH_KEY command through the IPCC mechanism (explained in the sections below)
3. Send FUS_GET_STATE until getting state equal to FUS_STATE_IDLE.
   This operation does not generate any system resets.

Once the user authentication key is installed, it can be changed (unless lock user authentication key operation is done) using the same flow as above. But it cannot be removed.

Once it is installed, FUS systematically checks on the binary user signature before performing the installation or upgrade. If the signature is not present or if it is not authentic, the install or upgrade is rejected with error equal to FUS_STATE_IMG_NOT_AUTHENTIC.

4.2 Lock user authentication key

FUS allows the user authentication key to be locked. It means that this key can no longer be changed for the entire product life cycle. There is no way to undo this operation once it is performed.

To lock the user authentication key:

1. Ensure that FUS is running (follow the steps in Section 1.3: How to activate FUS).
2. Send FUS_LOCK_AUTH_KEY with FUSv2 command through the IPCC mechanism (explained in the sections below).
3. Send FUS_GET_STATE util getting state equal to FUS_STATE_IDLE.
   This operation does not generate any system resets.

Once this operation is done, the user authentication key is locked.

5 Customer key storage

The FUS allows customer keys to be stored in the dedicated FUS flash memory area and then to load the stored key to the AES1 in secure mode (the AES1 key register is only accessed by Arm® Cortex®-M0+ and data registers accessible by Arm® Cortex®-M4 user application).

5.1 Key types and structure

FUS supports the storage of 101 keys (1 master key and 100 clear/encrypted keys).

Key size can be 128 bits or 256 bits. The key size and structure is the same for all types of keys. Any stored key cannot be changed or removed.

FUS supports three key types:

- **Clear key**: a key sent to FUS, unencrypted.
- **Master key**: a key sent to FUS, unencrypted and used to decrypt other keys to be sent to FUS later. The storage of this key must be done in a trusted environment (where the key cannot be extracted on the communication path). It allows the user to handle encrypted keys in untrusted environments without exposing the content. A master key cannot be written in the AES1 key register. It is exclusively used for decryption and cannot be changed or removed. The Master key is written only once and is never updated afterwards. Once the master key is written, any request to write the master key again is rejected with an error message. Writing more than 100 keys, results in the command being rejected.
- **Encrypted key**: a key that is sent to FUS in encrypted format. It is then decrypted by FUS using the master key before using it. This key must be accompanied by an IV (initialization vector) allowing its decryption by FUS. 12-Bytes IV is sent in the same command packet as the key itself.

Note: IV is 12 Bytes only and the complementary 4 bytes (to reach 16 bytes) are fixed by FUS to 0x00, 0x00, 0x00, 0x02.

The user key encryption must be based on AES-128 GCM mode. The FUS decrypts the key without using the AES hardware.

The key type must be communicated to FUS in the command packet where the key is sent (more details in commands description).

Keys are managed through their index.

When a key is sent to FUS, FUS acknowledges its reception and responds with the key allocated index. This index is assigned by FUS and cannot be changed by user application.

To store a key, the user application must send FUS_STORE_USR_KEY or FUS_CMD_WRITE_USER_KEY_IN_MEMORY for FUSv2 to FUS (with key type and the associated IV, if any) and then receive the key index.

To use the stored key, the user application must:

- Configure AES1 initialization registers and IV register.
- Send FUS_LOAD_USR_KEY or FUS_CMD_WRITE_USER_KEY_INTO_AES for FUSv2 to FUS and wait for the response to be received which means the key has been written in the AES1 key register.
- Write in AES1 data register to decrypt/encrypt data using the stored key (the key register remains protected and cannot be accessed by Arm® Cortex®-M4 user application). If more than 100 keys are written, it results in that command being rejected.

There are two additional services provided by FUS for user keys. These two services are intended for use by the Arm® Cortex®-M4 user application in the context of a secure application and they are not exposed by bootloader or STM32CubeProgrammer.

Note: Example of an encrypted key structure (payload):

- *Byte0: 0x03 (type=encrypted)*
- *Byte1: 0x20 (size of the key: 32 bytes, 256 bits)*
- *Byte2: KeyData[0]*
- *Byte3: KeyData[1]*
- *...*
- *Byte33: KeyData[31]*
- *Byte34: IV[0]*
- *Byte35: IV[1]*
- *Byte44: IV[10]*
- *...*
- *Byte45: IV[11]*

User key lock

This service ensures that a key can no longer be used by any application (cannot be loaded into AES) until the next device reset. It is possible to use this service by sending a FUS_LOCK_USR_KEY command containing the index of the key to be locked (Master key index is always 0 and it cannot be locked neither loaded).

When the FUS_LOCK_USR_KEY command is sent, FUS stores the state of the requested key as locked and issuing any FUS_LOAD_USR_KEY for that key index results in operation fail (0x01 returned by the command response).

User key unload

This service is used to unload the key currently loaded in AES (if FUS_LOAD_USR_KEY has been used) and prevent any further operation using the loaded key by the user application.

It is possible to use this service by sending a FUS_UNLOAD_USR_KEY command containing the index of the key to be unloaded (Master key index is always 0 and it cannot be loaded neither unloaded).

When FUS_UNLOAD_USR_KEY is sent, FUS writes zeros into the key registers of the AES and thus the loaded key cannot be used anymore.

6 Communication with FUS

Communication with FUS is performed through the IPCC channels and by Arm® Cortex®-M4 user application or by bootloader or by JTAG. In all cases, the communication principles are exactly the same.

Using STM32 system bootloader to communicate with FUS provides abstraction of all the low layers by directly using bootloader interfaces (USART or USB-DFU).

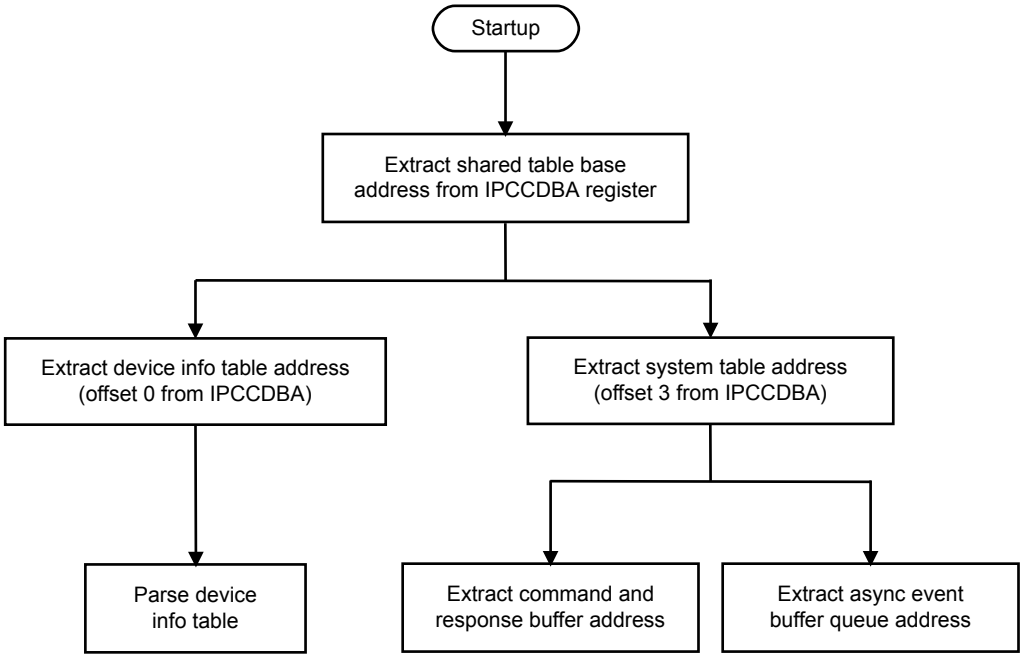
To communicate with the FUS, there are two elements to be used:

- Shared tables: used to store FUS information and to get the command/response packets.
- IPCC: used to exchange message notifications (message content is located in the shared tables).

6.1 Shared tables usage

Shared tables are an information structure located in SRAM2a public area, whose structure is explained in the following subsections. FUS uses two shared tables, namely the device information table, and the system table. Both of them must be parsed by the Arm® Cortex®-M4 user application (or JTAG application), to correctly communicate with FUS.

Figure 4. Shared table usage process



6.1.1 Device information table

This is a 42-byte buffer used to update the current status of the device. The table may be updated either by the wireless stack code or by the FUS, at startup or before a programmed system reset.

Table 9. Device information table

Field	Size (bytes)	Values
Device info table state	4	0xA94656B9: Device info table valid Any other value: Device info table is not valid
Reserved	1	Reserved
Last FUS active state	1	0x00: FUS idle 0x01: Wireless stack firmware upgrade 0x02: FUS firmware upgrade 0x03: FUS service 0x04: Wireless stack running 0x05-0xFE: Not used 0xFF: Error
Last wireless stack state	1	0x00: Not started 0x01: Running 0x08-0xFE: Not used 0xFF: Error
Current wireless stack type	1	0x00: None 0x01: Bluetooth® Low Energy 0x02: Thread type1 0x03: Thread type2 More details are available in the wireless stack documentation.
Safe boot version	4	Firmware version: [31:24]: Major (updated when backward compatibility is broken) [23:16]: Minor (updated when a major feature is added) [15:8]: Subversion (updated for minor changes) [7:4]: Branch (specific build) [3:0]: Build (build version)
FUS version	4	Firmware version [31:24]: Major (updated when backward compatibility is broken) [23:16]: Minor (updated when a major feature is added) [15:8]: Subversion (updated for minor changes) [7:4]: Branch (specific build) [3:0]: Build (build version)
FUS memory size	4	Current FUS stack memory usage: [31:24]: SRAM2b number of 1 K sectors used [23:16]: SRAM2a number of 1 K sectors used [15:8]: Reserved [14:0]: flash memory number of 4 K sectors used
Wireless stack version	4	Firmware version: [31:24]: Major (updated when backward compatibility is broken) [23:16]: Minor (updated when a major feature is added) [15:8]: Subversion (updated for minor changes) [7:4]: Branch (specific build) [3:0]: Build (build version) When no stack is present, all data is 0xFFFF FFFF

Field	Size (bytes)	Values
Wireless stack memory size	4	Current wireless stack memory usage: [32:24]: SRAM2b number of 1 K sectors used [23:16]: SRAM2a number of 1 K sectors used [15:8]: Reserved [14:0]: flash memory number of 4 K sectors used When no stack present, all data is 0xFFFF FFFF
Wireless FW-BLE info	4	[31:0]: Reserved for wireless stack usage When no stack present, all data is 0xFFFF FFFF

6.1.2 System table

The system table is an 8-byte table containing two buffer pointers, described in the table below.

Table 10. System table content

Address	Size (bytes)	Content	Description
0x00	4	Address of system command/response buffer	A single buffer is used at any given time, only a command or its response must be written. The response overwrites the command. The new command overwrites any previous command response.
0x04	4	Address of system events queue buffer (address of first event)	FUS code has to parse and fill the queue when necessary. Events messages are managed as a chained list and are freed once Arm® Cortex®-M4 has read them (notification through IPCC). Parsing of the event is done through their size only. (not chained list structure),

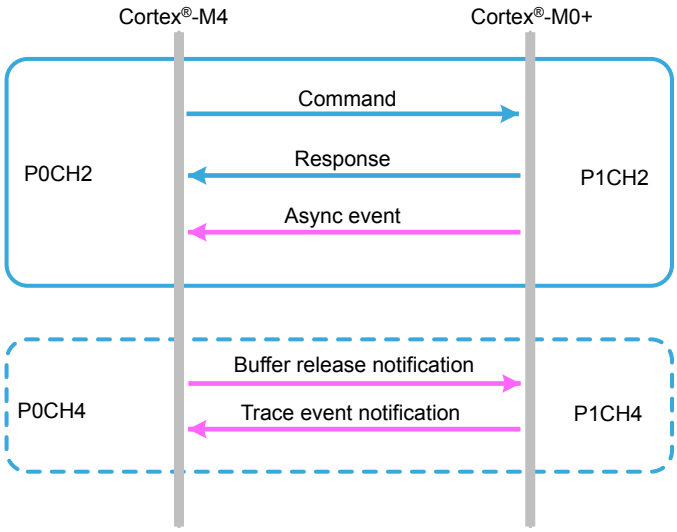
To get useful information to communicate with FUS, the Arm® Cortex®-M4 code (application or bootloader) performs parsing as described in Figure 4.

6.2 IPCC usage

FUS uses system IPCC allocated channels: P0CH2 (on Arm® Cortex®-M4 side) and P1CH2 (on Arm® Cortex®-M0+ side). These channels offer three communication ways:

- Cmd: Command request from Arm® Cortex®-M4 to Arm® Cortex®-M0+. This route is used to send a command to Arm® Cortex®-M0+.
- Rsp: Response to command from Arm® Cortex®-M0+ to Arm® Cortex®-M4. This route is used only to answer a command requested by Arm® Cortex®-M4.
- Asynch Evt: Asynchronous event from Arm® Cortex®-M0+ to Arm® Cortex®-M4. This route is used to inform Arm® Cortex®-M4 about an asynchronous event, without requiring an answer from Arm® Cortex®-M4 on this event.

Figure 5. **IPCC channels used by FUS**

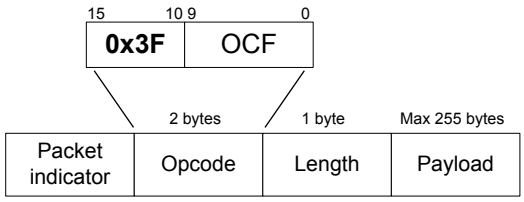


6.3 FUS commands

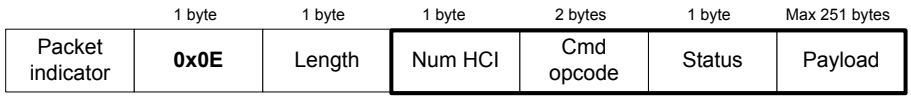
FUS uses the same command/response structure as wireless stacks and is based on the HCI model. FUS uses a subset of the HCI commands, namely:

- Vendor specific HCI command packet: used to send command from the Arm® Cortex®-M4 to the Arm® Cortex®-M0+.
- HCI command complete event packet: used to send a response from the Arm® Cortex®-M0+ to the Arm® Cortex®-M4
- Vendor specific HCI event packet: used to send asynchronous events from the Arm® Cortex®-M0+ to the Arm® Cortex®-M4.

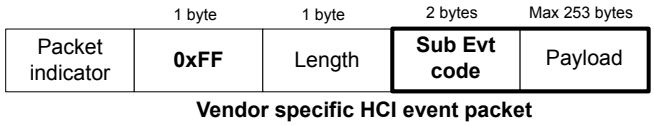
Figure 6. **FUS HCI subset**



Vendor specific HCI command packet



HCI command complete event packet



Vendor specific HCI event packet

6.3.1 Packet indicators

The packet indicator is one byte and its value depends on the packet type.

Table 11. Packet indicator values

Packet type	Packet indicator value
Command packet	0x10
Response packet	0x11
Event packet	0x12

6.3.2 Event packet

An asynchronous event is sent only once, at startup, by FUS.
The length field represents the length of SubEvtCode+Payload.

Table 12. FUS asynch event (vendor specific HCI event)

Length	SubEvtCode	Payload	Meaning
3	0x9200	Error code: • 0x00: Wireless stack running • 0x01: FUS running • 0x02: Software error • 0x03 to 0xFF: Not used	FUS initialization phase is done and the error code presented in the payload byte.

6.3.3 Command packet

Note: Prefix is FUS_CMD_xx for FUSv2.

The table below details all commands supported by FUS and their HCI format values.

Table 13. FUS commands (vendor specific HCI command packet)

Command	Opcode	Length (bytes)	Payload
Reserved	0xFC00	N/A	N/A
FUS_GET_STATE	0xFC52	0	None
Reserved	0xFC53	N/A	N/A
FUS_FW_UPGRADE	0xFC54	0 / 4(1) / 8 (2)	None (optional 4 bytes) address of the firmware image location (optional 8 bytes) address of the firmware destination
FUS_FW_DELETE	0xFC55	0	None
FUS_UPDATE_AUTH_KEY	0xFC56	Up to 65	Byte0: authentication key size N in bytes Byte1 to ByteN-1: authentication key data
FUS_LOCK_AUTH_KEY	0xFC57	0	None
FUS_STORE_USR_KEY	0xFC58	N+2+(Tx12)(3)	Byte0: key type: • 0x00:None • 0x01:Simple key • 0x02: Master key • 0x03: Encrypted key Byte1: key size N in bytes Byte2 to ByteN+1: key data (key value) ByteN+2 to ByteN+14: IV 12 bytes if the key type is Encrypted key

Command	Opcode	Length (bytes)	Payload
FUS_LOAD_USR_KEY	0xFC59	1	Byte0: key index (from 0 to 124)
FUS_START_WS	0xFC5A	0	None
FUS_LOCK_USR_KEY	0xFC5D	1	One byte, index of the key to be locked
FUS_UNLOAD_USR_KEY	0xFC5E	1	One byte, index of the key to be unloaded
FUS_ACTIVATE_ANTIROLLBACK	0xFC5F	0	None
Reserved	0xFC60 to 0xFCFF	N/A	N/A

1. *4 bytes, not used in the current version.*
2. *8 bytes, not used in the current version.*
3. *Where N= the size of the key in bytes and T=0 if the key is master or simple and T=1 if the key is encrypted.*

6.3.4 Response packet

For each command packet, a response packet is sent by FUS containing information detailed in table below. The NumHCI field value is always set to 0xFF.

The length field indicates the length of NumHCI+CmdOpcode+Status+Payload. So, if there is no payload, the length value is four.

Table 14. FUS responses (HCI command complete packet)

Status	Length	Cmd Opcode value	Status value	Payload
FUS_STATE	5	0xFC52	Values in table FUS state values	Values in table FUS state error values.
FW_UPGRADE_STATE	4	0xFC54	• 0x00: Operation started • 0x01: Fail • 0x02-0xFF: Not used	None
FW_DELETE_STATE	4	0xFC55		None
UPDATE_AUTH_KEY_STATE	4	0xFC56	• 0x00: Operation done • 0x01: Fail • 0x02-0xFF: Not used	None
LOCK_AUTH_KEY_STATE	4	0xFC57		None
STORE_USR_KEY_STATE	5	0xFC58		One byte: Stored key index (from 0 to 100)
LOAD_USR_KEY_STATE	4	0xFC59		None
FUS_START_WS_START	4	0xFC5A	• 0x00: Operation started • 0x01: Fail • 0x02-0xFF: Not Used	None
FUS_LOCK_USR_KEY	4	0xFC5D	• 0x00: Operation done • 0x01: Fail • 0x02-0xFF: Not used	None
FUS_UNLOAD_USR_KEY	4	0xFC5E	• 0x00: Operation done • 0x01: Fail • 0x02-0xFF: Not used	None
FUS_ACTIVATE_ANTIROLLBACK	4	0xFC5F	• 0x00: Operation done • 0x01: Fail • 0x02-0xFF: Not used	None

FUS response state values are detailed in the table below. Some values are represented as a range (for example 0x10 to 0x1F), which means all values from that range represent the same state meaning (for example 0x12 or 0x1E both mean FUS_STATE_FW_UPGRD_ONGOING). This range of values is reserved for future extensions of the protocol.

Table 15. FUS state values

Value	Name	Meaning
0x00	FUS_STATE_IDLE	FUS is in idle state. Last operation was done successfully and returned its state. No operation is ongoing.
0x01..0x0F	Not used	These values are reserved for future use.
0x10..0x1F	FUS_STATE_FW_UPGRD_ONGOING	The firmware upgrade operation is ongoing.
0x20..0x2F	FUS_STATE_FUS_UPGRD_ONGOING	The FUS upgrade operation is ongoing.
0x30..0x3F	FUS_STATE_SERVICE_ONGOING	A service is ongoing: Authentication key service (update/lock) or user key service (store/load).
0x40..0xFE	Not Used	These values are reserved for future use.
0xFF	FUS_STATE_ERROR	An error occurred. For more details about the error origin, refer to the response payload.

Table 16. FUS state error values

Value	Name	Meaning
0x00	FUS_STATE_NO_ERROR	No error occurred.
0x01	FUS_STATE_IMG_NOT_FOUND	Firmware/FUS upgrade requested but no image found. (such as image header corrupted or flash memory corrupted).
0x02	FUS_STATE_IMG_CORRUPTED	Firmware/FUS upgrade requested, image found, authentic but not integer (corruption on the data).
0x03	FUS_STATE_IMG_NOT_AUTHENTIC	Firmware/FUS upgrade requested, image found, but its signature is not valid (wrong signature, wrong signature header).
0x04	FUS_SATE_NO_ENOUGH_SPACE	Firmware/FUS upgrade requested, image found and authentic, but there is no enough space to install it due to the already installed image. Install the stack in a lower location then try again.
0x05	FUS_IMAGE_USRABORT	Operation aborted by user or power off occurred.
0x06	FUS_IMAGE_ERSERROR	Flash Erase Error
0x07	FUS_IMAGE_WRTERROR	Flash Write Error
0x08	FUS_AUTH_TAG_ST_NOTFOUND	STMicroelectronics Authentication tag not found error in the image.
0x09	FUS_AUTH_TAG_CUST_NOTFOUND	Customer Authentication tag not found in the image.
0x0A	FUS_AUTH_KEY_LOCKED	The key that the user tries to load is currently locked.
0x11	FUS_FW_ROLLBACK_ERROR	Rollback to older version of FW detected and not allowed.
0x12	FUS_BUSY	There is an operation already ongoing.
0x13	FUS_INVALID_CKS_OPERATION	CKS operation not allowed.
0x14...0xFD	N/A	Reserved for future use.
0xFE	FUS_STATE_NOT_RUNNING	FUS is not currently running. wireless stack is running and returned this state.
0xFF	FUS_STATE_ERR_UNKOWN	Unknown error

6.4 Image footers

Each element of the image has its own footer:

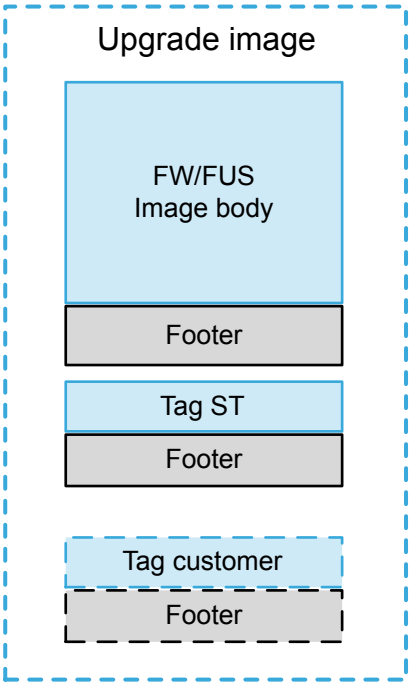
- The image body
- The STMicroelectronics signature (mandatory element)
- The customer signature (optional element)

The footers must follow on directly from the end of their relative element (for example, the image body header address must be contiguous to the image body address).

The authentication tags do not have this continuity obligation. They do not need to be located next to the image. They can be located anywhere in the user flash memory. FUS looks for them independently of the image location.

All images, footers addresses, and sizes must be multiples of four bytes, or FUS cannot identify them.

Figure 7. Image footers placement



Each footer contains an identification value allowing FUS to recognize it.

Table 17. FW/FUS upgrade image footer structure

Info1[1]	Reserved								Reserved								NVM section size								Reserved						
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Info2[2]	Reserved																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Memory size	SRAM2b[3]								SRAM2a [3]								Footer type								Flash[4]							
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Version	Version major								Version minor								Subversion								Branch build							
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Magic number	Magic number																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

1. *For example, Bluetooth® LE*
2. *For example, thresd*
3. *Number of 1 K sectors*
4. *Number of 4 K sectors*

Table 18. Parsing of image footer structure

Field	Meaning
Nvm section size	Size of the saved NVM section with pairing data (only for footer type 2)
Flash memory	Image total size expressed as a multiple of 4 Kbytes
Footer type (can be either 1 or 2)	1. - 0xFF: footer type 1, pairing data migration feature not supported 2. - 0x02: footer type 2, pairing data migration feature supported
SRAM2a	Image total required space in SRAM2a secure area
SRAM2b	Image total required space in SRAM2b secure area
Build	Version build number
Branch	Version branch number
SubVersion	Version subversion number
VersionMinor	Version minor number
VersionMajor	Version major number
Magic Number	Specific value allowing to identify the nature of the image.

Note: FUS V1.2.0 version is written in the binary as 0xFFFFFFFF to be able to upgrade from all versions of FUS.

Figure 8. Signature (tag) footer structure

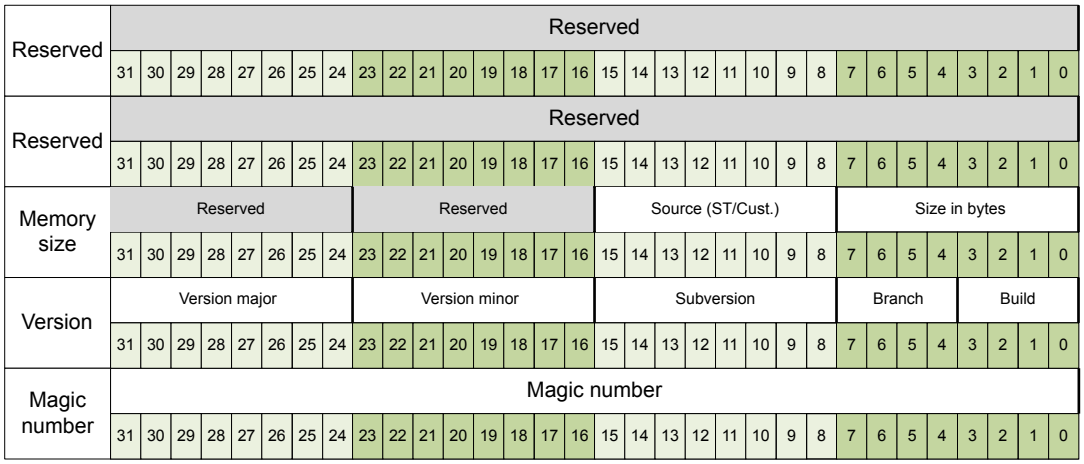


Table 19. Parsing of signature footer

Field	Meaning
Reserved	Not used in this version
Size	Signature total size in bytes (without footer)
Source	Signature nature: 0x00: ST signature 0x01: Customer signature
Build	Version build number
Branch	Version branch number
SubVersion	Version subversion number
VersionMinor	Version minor number
VersionMajor	Version major number
Magic Number	Specific value allowing to identify the nature of the image.

The magic number values allowing to identify the image nature are detailed in the table below:

Table 20. Magic number values

Value	Nature
0x23372991	Wireless stack image (legacy for footer type 1)
0xB10C8B99	Wireless stack image (only for footer type 2)
0x32279221	FUS Image
0xD3A12C5E	STMicroelectronics signature
0xE2B51D4A	Customer signature
0x42769811	Other firmware image
0x927A6741	Safeboot image update (v2.0.1, only compliant with FUSv2)

7 STM32 system bootloader extension for FUS

A command set extension has been added to the STM32WB system bootloader to support FUS operation. These commands are implemented on USART and USB-DFU interfaces and follow the same rules as existing standard bootloader commands.

To help to understand this section, a prior reading of *STM32 microcontroller system memory boot mode* (AN2606) and *USART protocol used in the STM32 bootloader* (AN3155) and *USB DFU protocol used in the STM32 bootloader* (AN3156) documentation is required.

7.1 USART extension

Two commands have been added to the bootloader USART standard protocol to support the FUS extension. All FUS commands are passed through these two special commands: one for writing (used for all FUS commands from host to FUS) and one for reading (used for all FUS commands from FUS to host).

Table 21. Bootloader USART command extension

Command	Opcode	Usage
Special read command	0x50 (complement 0xAF)	Get data from FUS
Special write command	0x51 (complement 0xAE)	Send data to FUS

Note: For the bootloader, the following commands added in FUS are not supported (neither on UART nor USB DFU).

- *FUS_LOCK_USR_KEY*
- *FUS_UNLOAD_USR_KEY*
- *FUS_ACTIVATE_ANTIROLLBACK*

Lock and Unload user key are two commands that are meant for use by Arm® Cortex®-M4 user applications only.

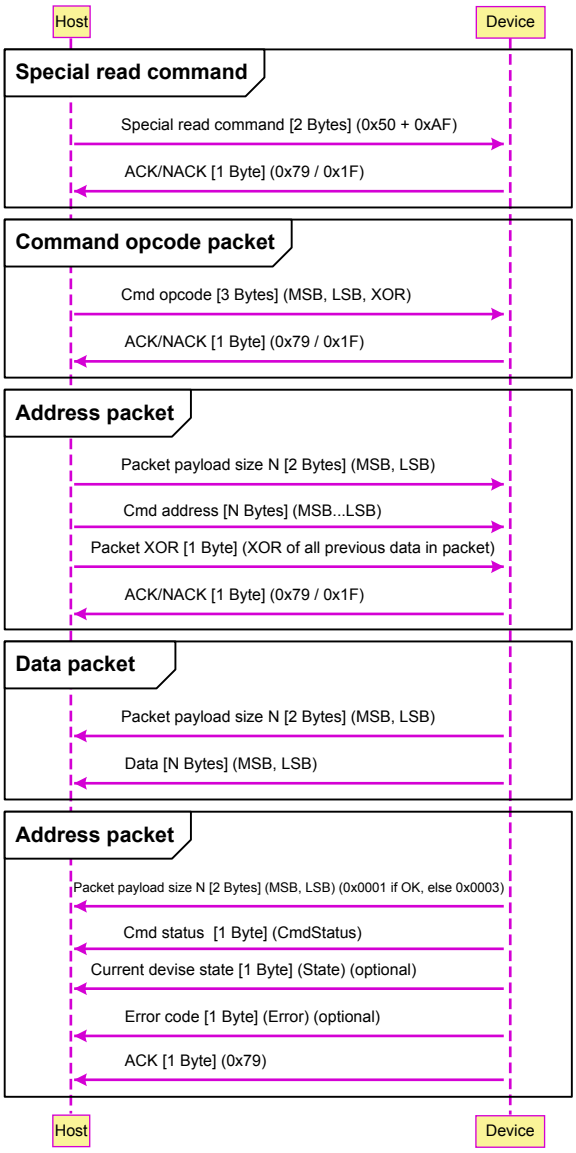
Activate anti-rollback can be used either by implementing it in Arm® Cortex®-M4 user application code, or by using STM32CubeProgrammer features or by using STM32 open bootloader example code.

7.1.1 USART special read

A special read command is used to perform the FUS command management. It is divided into five separate packets:

- Special read command packet:
  - The host sends the special read command code and complement (0x50, 0xAF) and waits for the ACK/NACK byte. In the case of NACK, it means that the command is not supported.
- Command opcode packet
  - The host sends the command packet containing:
    - FUS command opcode (2 bytes)
    - XOR of the FUS command opcode (2 bytes)
  - The device sends ACK if the opcode is supported. NACK otherwise.
- Address packet:
  - The host sends the address packet payload size on two bytes (MSB first).
  - Host sends address payload bytes (MSB first).
  - Host sends packet XOR value (checksum of all previous bytes in current packet, 1 byte).
  - The device sends ACK if data is correct and supported. NACK otherwise.
- Response data packet: (Optional)
  - The device sends the packet data payload size in bytes on 2 bytes (MSB first).
  - The device sends data payload bytes (MSB first). Some commands require not data payload.
- Response status packet:
  - The device sends the packet payload size in bytes on 2 bytes (MSB first).
  - The device sends the command status on one byte (status of the current command requested by the host).
  - The device sends the current device state on 1 byte (optional, if payload size > 3).
  - The device sends the current command error code (or key index) on 1 byte.
  - The device sends ACK to signal end of the response packet.

Figure 9. **USART special read command**

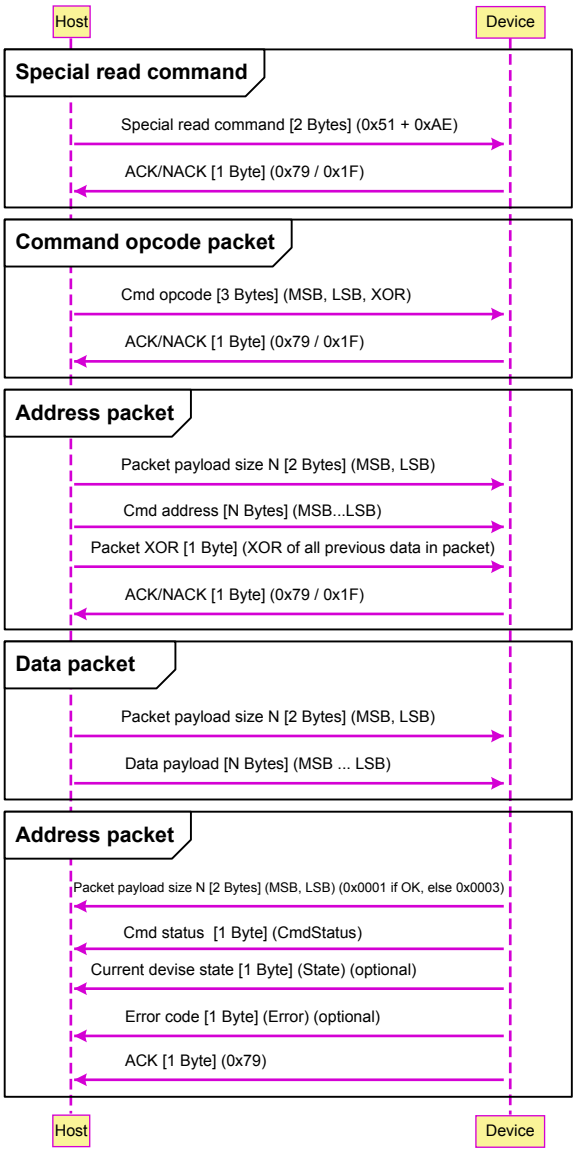


7.1.2 USART special write

A special write command is used to perform the FUS command sending requesting data from the device. It is divided into four separate packets:

- Special write command packet: the host sends the special write command code and complement (0x51, 0xAE), and waits for the ACK/NACK byte. In the case of NACK, it means that the command is not supported.
- Command opcode packet:
  - The host sends the command packet containing:
    - FUS command opcode (2 bytes)
    - XOR of the FUS command opcode (2 bytes)
  - The device sends ACK if the opcode is supported. NACK otherwise.
- Address packet:
  - The host sends the address packet payload size on two bytes (MSB first).
  - Host sends address payload bytes (MSB first).
  - Host sends packet XOR value (checksum of all previous bytes in current packet, 1 byte).
  - The device sends ACK if data is correct and supported. NACK otherwise.
- Data packet:
  - The host sends packet data payload size in bytes on 2 bytes (MSB first). This number may be zero when no data is needed for the command.
  - The host sends data payload bytes (MSB first). No data is sent if the payload size is zero.
  - Host sends packet XOR value (checksum of all previous bytes in current packet, 1 byte).
  - The device sends ACK if the data is correct and supported. NACK otherwise.
- Response packet:
  - The device sends the packet payload size in bytes on 2 bytes (MSB first).
  - The device sends command status on one byte (status of the current command requested by the host).
  - The device may send the current device state on 1 byte (optional, if payload size > 1).
  - The device sends current command error code on 1 byte (optional, if payload size > 1).
  - The device sends ACK to signal end of the response packet.

Figure 10. **USART special write command**



7.1.3 USART FUS command mapping

There is only one FUS command mapped on special read command.

Table 22. USART FUS command mapping on a read command

Command	Opcode	Address packet	Data packet	Cmd status packet
FUS_GET_STATE	0x54	Size = 0x0000 Data = None	Size = 0x0003 Data = [0x00, FUS_STATE, ErrorCode]	Size = 0x0001 or 0x0003 Data = [0x00] if OK or [0x01, state, error] if KO

There are seven FUS commands mapped on a special write command.

Table 23. USART FUS command mapping on write command

Command	Opcode	Address packet	Data packet	Cmd status packet
FUS_FW_DELETE	0x52	Size = 0x0000 Data = None	Size = 0x0000 Data = None	Size = 0x0001 or 0x0003 Data = [0x00] if OK or [0x01, state, error] if KO
FUS_FW_UPGRADE	0x53	Size = 0x0000 Data = None	Size = 0x0000 Data = None	
FUS_UPDATE_AUTH_KEY	0x56	Size = 0x0000 Data = None	Size = up to 65 Data = Key (1 byte key size + 64 bytes key data)	
FUS_LOCK_AUTH_KEY	0x57	Size = 0x0000 Data = None	Size = 0x0000 Data = None	
FUS_STORE_USR_KEY	0x58	Size = 0x0000 Data = None	Size = up to 34 Data = [KeyType (1byte), KeySize(1byte), KeyData (16/32bytes)]	Size = 0x0003 Data = [0x00, state, KeyIndex]
FUS_LOAD_USR_KEY	0x59	Size = 0x0000 Data = None	Size = 0x0001 Data = [KeyIndex]	Size = 0x0001 or 0x0003 Data = [0x00] if OK or [0x01, state, error] if KO
FUS_START_WS	0x5A	Size = 0x0000 Data = None	Size = 0x0000 Data = None	

7.2 USB-DFU extension

FUS commands are processed over bootloader USB-DFU standard download and upload commands.

7.2.1 USB-DFU download FUS extension

Bootloader USB-DFU download FUS extension is managed in the same way as SET_ADDRESS_POINTER and ERASE standard commands: Value = 0 and the following bytes are command data MSB first.

Exception is made for FUS_STORE_USR_KEY, which is split over two steps:

1. Download command, only allows to send the key data (up to 34 bytes)
2. Upload command must be done after the download step and allows to get the key index (1 byte)

Table 24. **USB-DFU download extension**

Command	Opcode	Data
FUS_FW_DELETE	0x52	None
FUS_FW_UPGRADE	0x53	None
FUS_UPDATE_AUTH_KEY	0x56	Key buffer = [KeySize (1byte), KeyData (64bytes MSB first)]
FUS_LOCK_AUTH_KEY	0x57	None
FUS_STORE_USR_KEY	0x58	Key Buffer = [KeyType (1byte), KeySize(1byte), KeyData (16/32bytes)]
FUS_LOAD_USR_KEY	0x59	Key Index (1byte)
FUS_START_WS	0x5A	None

7.2.2 USB-DFU upload FUS extension

Bootloader USB-DFU upload FUS extension is managed in the same way as the regular upload command for reading physical address (wBlockNum > 1). But in this case, a virtual memory address mask is used: 0xFFFF0000. So, the FUS read command is managed through a read to virtual address 0xFFFF00YY where YY is the FUS command opcode.

Upload command allows to perform the second step of FUS_STORE_USR_KEY, which is getting the key index.

Table 25. **USB-DFU upload extension**

Command	Address	Returned data
FUS_GET_STATE	0xFFFF0054	State buffer = [FUS state (1byte), FUS error code (1byte)]
FUS_STORE_USR_KEY	0xFFFF0058	Key index (1byte)

8 FAQ and troubleshooting

Table 26. Frequently asked questions and answers

Question	Answer
When I receive a virgin STM32WB device from STMicroelectronics, what does it contain exactly?	All STM32WB devices delivered by STMicroelectronics contain by default the FUS and the bootloader. They do not contain any preinstalled wireless stacks.
I cannot read the FUS version	Accessing the device information table is possible when the following conditions are met: 1. Device info table address is written in location pointed by the IPCCDBA option byte. 2. Arm® Cortex®-M0+ is enabled 3. FUS is running on Arm® Cortex®-M0+ (and not wireless stack) (If the wireless stack is running, it is possible to force FUS to run by sending 2 FUS_GET_STATE commands). So, when accessing a device via SWD, it is normal not to have a valid device info table, because it has not yet been written or Arm® Cortex®-M0+ has not been enabled yet. That is why it is more convenient to read the device info table when the bootloader is running because it performs the actions (1) and (2) above. *Note: It is possible to connect through SWD and disable the hardware reset option (hot plug) and keep boot on bootloader that allows the user to read the device info table.*
I want to upgrade the FUS image and I already have a wireless stack installed. Do I need to delete the wireless stack prior to upgrading FUS?	It is advised to delete the wireless stack before performing the FUS upgrade in general and especially when upgrading from FUS V0.5.3. If the existing FUS version is higher than V0.5.3, then it is not mandatory to perform the wireless stack deletion.
How do I know quickly if my device is running FUS or wireless stack?	There are multiple ways to check it: • Read the Option Bytes and check the value of SBRV. If FUS is running, it is 0x3D000 (or 0x3D800 if FUS V0.5.3 is running) • Read the device information table @0x20030030, if it is different for the FUS version, then the wireless stack is running or Arm® Cortex®-CM0+ is not enabled. • Send FUS_GET_STATE command, if FUS_STATE_NOT_RUNNING is received, then the wireless stack is running or Arm® Cortex®-CM0+ is not enabled.
What is IPCCDBA Option Byte used for?	IPCCDBA is used to change the offset where to read/write the device information table.
After an upgrade operation, I cannot access flash memory anymore and cannot communicate with FUS.	First check if SFSA=0x00. If it is the case, then it means the safeboot has been triggered. Safeboot is triggered when an Option Bytes corruption occurs. This may occur during a FUS upgrade operation or during any user application operation dealing with Option Bytes. When safeboot is triggered, it locks the device by setting SFSA=0x00 (all flash memory secure) and so no user application/debugger can access the user flash memory anymore. This operation is not reversible. Starting from FUS v1.1.0, the safeboot is modified to perform a factory reset instead of locking the device.
Is it possible to downgrade the FUS version (for example when the current FUS version running is v1.0.2, is it possible to install FUS v1.0.1?)	FUS downgrade is not possible in any combination. It can only be upgraded to higher or equal versions. In the case of a tentative downgrade, FUS simply rejects the upgrade and returns an error message.

Question	Answer
What is a typical STM32CubeProgrammer command to perform an upgrade using FUS?	1. First check that FUS is running by sending FUS_GET_STATE commands until receiving a FUS_STATE_IDLE state response: – STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate – STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate – STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate Sending 3 times the FUS_GET_STATE command ensures that FUS is running and idle in most cases. 2. Delete the existing wireless stack and install the new one (case of wireless stack upgrade): – STM32_Programmer_CLI.exe -c port=usb1 -fwupgrade stm32wb5x_BLE_Stack_fw.bin 0x080CB000 firstinstall=0 – STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate – STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate – ... (keep sending -fusgetstate until the received state is FUS_STATE_NOT_RUNNING) Setting "firstinstall=0" ensures that the previous stack is deleted before the new one is installed. Even if there is no previously installed stack, setting "firstinstall=0" would not cause any problem. Alternately proceed to FUS image installation (case of FUS upgrade): • STM32_Programmer_CLI.exe -c port=usb1 -fwupgrade stm32wb5x_FUS_fw.bin 0x080EC000 firstinstall=0 • STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate • STM32_Programmer_CLI.exe -c port=usb1 -fusgetstate • ... (keep sending -fusgetstate till received state is FUS_STATE_IDLE) "firstinstall=0" means the existing wireless stack is deleted prior to upgrading FUS. It is possible to use "firstinstall=1" if upgrading from FUS version different from FUS v0.5.3.
What is a safeboot and how can it be used?	Safeboot is an independent part of the FUS that manages specifically one case: option bytes corruption. When option bytes are corrupted, the STM32WB hardware forces the boot to safeboot whatever the running firmware. The safeboot then either: • Locks the device in full secure mode (on FUS versions lower than v1.1.0) which means all the device flash memory cannot be accessed and this operation is not reversible (there is no mean to cancel it and the device cannot be used anymore). • or performs a factory reset (on FUS versions v1.1.0 and higher) which means the wireless stack is removed if any and the Arm® Cortex®-M4 code is erased and boot is reset to FUS (virgin part state). This operation is also not reversible. To activate the Safeboot, the user must activate Arm® Cortex®-M0+ by writing the value 0x00008000 at the address 0x5800040C using the SWD interface.
Does FUS erase the shadow of encrypted firmware after installation?	Yes, FUS does erase the shadow remaining sectors of the encrypted firmware after it has been installed and moved to the upper address.
Is there a restriction on firmware image sizes that can be installed? Is it necessary to delete the installed firmware image before installing a new image?	When using FUS version older than V1.2.0: • If a wireless firmware image B is installed while another wireless firmware image A is already installed/running. if B size is larger than A size, and if B is loaded at an address too close to A (no enough free space between start address of A and end address of B, as explained in Section 2: Wireless stack image operations) then the device might be blocked with SBRV value pointing to the firmware image A (which is then corrupted) instead of pointing to firmware B, and the recovery might not be possible in that case. • For this, it is advised to delete the firmware image A before installing the firmware image B (in case of FOTA, this might be nonfeasible) or to make sure enough space is available before performing the installation. This known limitation is fixed in FUS V1.2.0.

Question	Answer
After upgrade of the FUS V1.2.0, I get an error message on STM32CubeProgrammer (or other programming interface) with the error code: "FUS_UFB_CORRUPT. What does it mean and what to do in this case?	When upgrading from some older versions of FUS to FUS V1.2.0, it is normal to have the error message FUS_UFB_Corrupt. It means that the UFB area (used for storing FUS configuration information) has been erased and needs to be configured to reset values. FUS then write the reset values and provoke a system reset and then the error is cleared. FUS returns FUS_IDLE state with no errors. No operation is required from the user side.
What happens if a firmware upgrade is started (for wireless stack or for FUS) and then interrupted by a forced system reset (from user or from CM4 code) or by a power loss? If this causes an issue, is it recoverable and how to recover it?	If a power loss or system reset is forced during the firmware upgrade, depending on which phase of the upgrade was interrupted: • In some cases, the operation can be automatically recovered by the FUS with no intervention from the user. • In some other cases, the firmware upgrade cannot be recovered. • – In this situation, erase all the flash memory sectors that are below the address indicated by the SFSA option byte and try to power off/on, to recover FUS operation. – You can also try to set the RDP level 1 then reset it to RDP level 0 to check if the option bytes are reloaded correctly. Also, erase any shadow incomplete chunks in the flash. If the secure flash memory area is corrupted during the reset/power loss, then the part might be unrecoverable. If the option bytes are corrupted, then the part might also be unrecoverable. A part is unrecoverable if, for example, SFSA=0x00 (and safeboot did not recover despite having the CM0 activated as described above) or if the FUS does not answer to any command despite that the CM0 is activated, IPCCDBA is set to the correct value, and the SharedTable content (that is, 0x20030000) remains inconsistent (that is, does not contain FUS version @0x20030030). *Note: Power supply instability might also lead to similar behavior and issues.*
What happens after attempting to install FUS v1.2 on a STM32WB that is already running FUSv2.x.y	This example applies to the use of the STM32WB5xxG (1M). For other memory densities, addresses may vary accordingly. Conditions: FUSv2.2.0 (or generally FUSv2.x.y) is installed, with SFSA = 0xF4. • An attempt to install FUS v1.2 at address 0x080EC000 fails as expected due to the FUS anti-rollback protection. After this update attempt, no erase occurs and the FUS v1.2 binary remains stored in memory in the range 0x080EC000 -0x080F3FFF. • The recommended installation address for stm32wb5x_BLE_Stack_full_fw.bin (v1.24.0, FUSv2 only) is 0x080D0000, as there may also be NVM data section between 0x080F0000 and 0x080F4000. It is not possible to install stm32wb5x_BLE_Stack_full_fw.bin at any address higher than 0x080D0000. • Installation of stm32wb5x_BLE_Stack_full_fw.bin (v1.24.0, FUS_v2 only), size 0x4000, erases sectors 208 - 239 (0x080D0000 - 0x080EEFFF). As area from 0x080F0000 to 0x080F4000 still contains part of the FUS v1.2 binary, the installed FUS v2.2 detects remaining FUS v1.2 binary and tries to install it, causing stm32wb5x_BLE_Stack_full_fw.bin installation process to fail. Solutions: • Perform a mass erase or • Erase the memory area from 0x080F0000 to 0x080F4000 (with SFSA = 0xF4 expected before the erase).

Revision history

Table 27. **Document revision history**

Date	Version	Changes
21-Mar-2019	1	Initial release.
17-Jun-2019	2	Added Section 1.2 FUS versioning and identification Updated: • Section 2 Wireless stack image operations, Section 5.1 Key types and structure, Section 5.1 Key types and structure • Table 20. USART FUS command mapping on write command, Table 22. USB-DFU upload extension
10-Jul-2019	3	Updated Table 7. Device information table
31-Mar-2020	4	Updated: • Section 1.1 Firmware upgrade services definition, Section 2 Wireless stack image operations, Section 2.1 Wireless stack install and upgrade, Section 2.2 Wireless stack delete, Section 5.1 Key types and structure, Section 7.1 USART extension • Table 1. FUS versions, Table 11. FUS commands (vendor specific HCI command packet), Table 12. FUS responses (HCI command complete packet), Table 14. FUS state error values Added: Section 2.4 Anti-rollback activation and Section 8 FAQ and troubleshooting
06-May-2021	5	Updated: • Section 1.2 FUS versioning and identification • Table 1. FUS versions • Section 1.3 How to activate FUS • Figure 1. Flash memory mapping • Section 1.5 FUS resources usage • Table 5. FUS resources usage • Section 2.4 Anti-rollback activation • Section 8 FAQ and troubleshooting Added: • Table 1. FUS versions • Table 2. FUS Versions Compatibility
22-Oct-2021	6	Updated: • Section 1.5 FUS resources usage • Figure 8. FW/FUS upgrade image footer structure • Figure 9. Signature (tag) footer structure • Section 8 FAQ and troubleshooting with new limitation
2-Aug-2022	7	Added Section 1.2.1 Known limitations Updated: • Section 2.1 Wireless stack install and upgrade • Section 2.2 Wireless stack delete • Section 8 FAQ and troubleshooting
16-Jan-2023	8	Updated • Table 5. FUS resources usage • Section 2.2 Wireless stack delete. Minor text edits across the whole document.
05-June-2023	9	Updated: • Section 5.1 Key types and structure • Table 11. FUS commands (vendor specific HCI command packet) • Table 23. Frequently ask and answer
10-Aug-2023	10	Updated: • Table 4. FUS activation cases. • Table 7. FUS resource usage. Minor text and terminology edits across the whole document.

Date	Version	Changes
30-Apr-2024	11	Updated: • Document title. • Section Introduction • Section 1.1: Firmware upgrade services definition • Section 1.2.1: Known limitations • Figure 2. SRAM memory mapping • Section 1.5: FUS resource usage • Section 1.6: Shared tables and memory usage • Section 2: Wireless stack image operations • Section 2.1: Wireless stack install and upgrade • Section 2.2: Wireless stack delete • Section 2.3: Wireless stack start • Section 3.2: Considerations on memory • Section 4.1: Install user authentication key • Section 5.1: Key types and structure • Section 6.1.1: Device information table • Section 6.3.2: Event packet • Section 6.3.4: Response packet • Section 6.4: Image footers • Section 7.1.1: USART special read • Section 8: FAQ and troubleshooting
11-Jul-2025	12	Added: • FUSv2 version • NVM section for migrated Bluetooth® LE pairing and bonding data • New FUS version v2.1.0 that mainly covers: – Robustness enhancements – Pairing/bonding data preservation (NVM) while FW upgrade is processed • All types of coprocessor binaries can be installed with this new version. New Safeboot version v2.0.1 with robustness enhancements. Updated: • Section 1: General information • Section 1.2: FUS versioning and identification • Section 1.3: How to activate FUS • Section 2.1: Wireless stack install and upgrade • Section 2.2: Wireless stack delete • Section 4.1: Install user authentication key • Section 4.2: Lock user authentication key • Section 5.1: Key types and structure • Table 9. Device information table • Table 16. FUS state error values • Section 6.4: Image footers • Section 6.2: IPCC usage • Table 20. Magic number values
10-Oct-2025	13	Added: • New FUS version v2.2.0 that covers enhanced security, SA0024 remains applicable
01-Nov-2025	14	Updated: • Section 1: General information • Section 1.5: FUS resource usage
13-Apr-2026	15	Updated: • Section 1.2: FUS versioning and identification • Section 1.4: Memory mapping • Section 2.2: Wireless stack delete • Table 20. Magic number values • Section 8: FAQ and troubleshooting

Contents

1 **General information** 2
1.1 Firmware upgrade services definition 2
1.2 FUS versioning and identification 3
1.2.1 Known limitations 5
1.3 How to activate FUS 5
1.4 Memory mapping 7
1.5 FUS resource usage 10
1.6 Shared tables and memory usage 12
2 **Wireless stack image operations** 13
2.1 Wireless stack install and upgrade 13
2.2 Wireless stack delete 15
2.3 Wireless stack start 15
2.4 Anti-rollback activation 15
3 **FUS upgrade** 17
3.1 Operation instructions 17
3.2 Considerations on memory 17
4 **User authentication** 18
4.1 Install user authentication key 18
4.2 Lock user authentication key 18
5 **Customer key storage** 19
5.1 Key types and structure 19
6 **Communication with FUS** 21
6.1 Shared tables usage 21
6.1.1 Device information table 21
6.1.2 System table 23
6.2 IPCC usage 23
6.3 FUS commands 24
6.3.1 Packet indicators 25
6.3.2 Event packet 25
6.3.3 Command packet 25
6.3.4 Response packet 26
6.4 Image footers 28
7 **STM32 system bootloader extension for FUS** 31
7.1 USART extension 31

7.1.1 USART special read 31
7.1.2 USART special write 34
7.1.3 USART FUS command mapping. 36
7.2 USB-DFU extension 36
7.2.1 USB-DFU download FUS extension 36
7.2.2 USB-DFU upload FUS extension 37
8 FAQ and troubleshooting 38
Revision history 41

List of tables

Table 1. FUS versions . . . 3
Table 2. FUS version compatibility . . . 4
Table 3. FUS versions availability . . . 5
Table 4. FUS activation cases . . . 6
Table 5. Main characteristics difference between STM32WB5x/WB3x and STM32WB1x with respect to FUSv2 . . . 8
Table 6. Additional device characteristics for different STM32WB flash derivatives with respect to FUSv2 . . . 9
Table 7. FUS resource usage . . . 10
Table 8. FUS upgrade returned errors . . . 14
Table 9. Device information table . . . 22
Table 10. System table content . . . 23
Table 11. Packet indicator values . . . 25
Table 12. FUS asynch event (vendor specific HCI event) . . . 25
Table 13. FUS commands (vendor specific HCI command packet) . . . 25
Table 14. FUS responses (HCI command complete packet) . . . 26
Table 15. FUS state values . . . 27
Table 16. FUS state error values . . . 27
Table 17. FW/FUS upgrade image footer structure . . . 29
Table 18. Parsing of image footer structure . . . 29
Table 19. Parsing of signature footer . . . 30
Table 20. Magic number values . . . 30
Table 21. Bootloader USART command extension . . . 31
Table 22. USART FUS command mapping on a read command . . . 36
Table 23. USART FUS command mapping on write command . . . 36
Table 24. USB-DFU download extension . . . 37
Table 25. USB-DFU upload extension . . . 37
Table 26. Frequently asked questions and answers . . . 38
Table 27. Document revision history . . . 41

List of figures

Figure 1. Flash memory mapping . . . 7
Figure 2. SRAM memory mapping . . . 8
Figure 3. Shared table architecture . . . 12
Figure 4. Shared table usage process . . . 21
Figure 5. IPCC channels used by FUS . . . 24
Figure 6. FUS HCI subset . . . 24
Figure 7. Image footers placement . . . 28
Figure 8. Signature (tag) footer structure . . . 30
Figure 9. USART special read command . . . 33
Figure 10. USART special write command . . . 35

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries (“ST”) reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice.

In the event of any conflict between the provisions of this document and the provisions of any contractual arrangement in force between the purchasers and ST, the provisions of such contractual arrangement shall prevail.

The purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST’s terms and conditions of sale in place at the time of order acknowledgment.

The purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of the purchasers’ products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

If the purchasers identify an ST product that meets their functional and performance requirements but that is not designated for the purchasers' market segment, the purchasers shall contact ST for more information.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2026 STMicroelectronics – All rights reserved